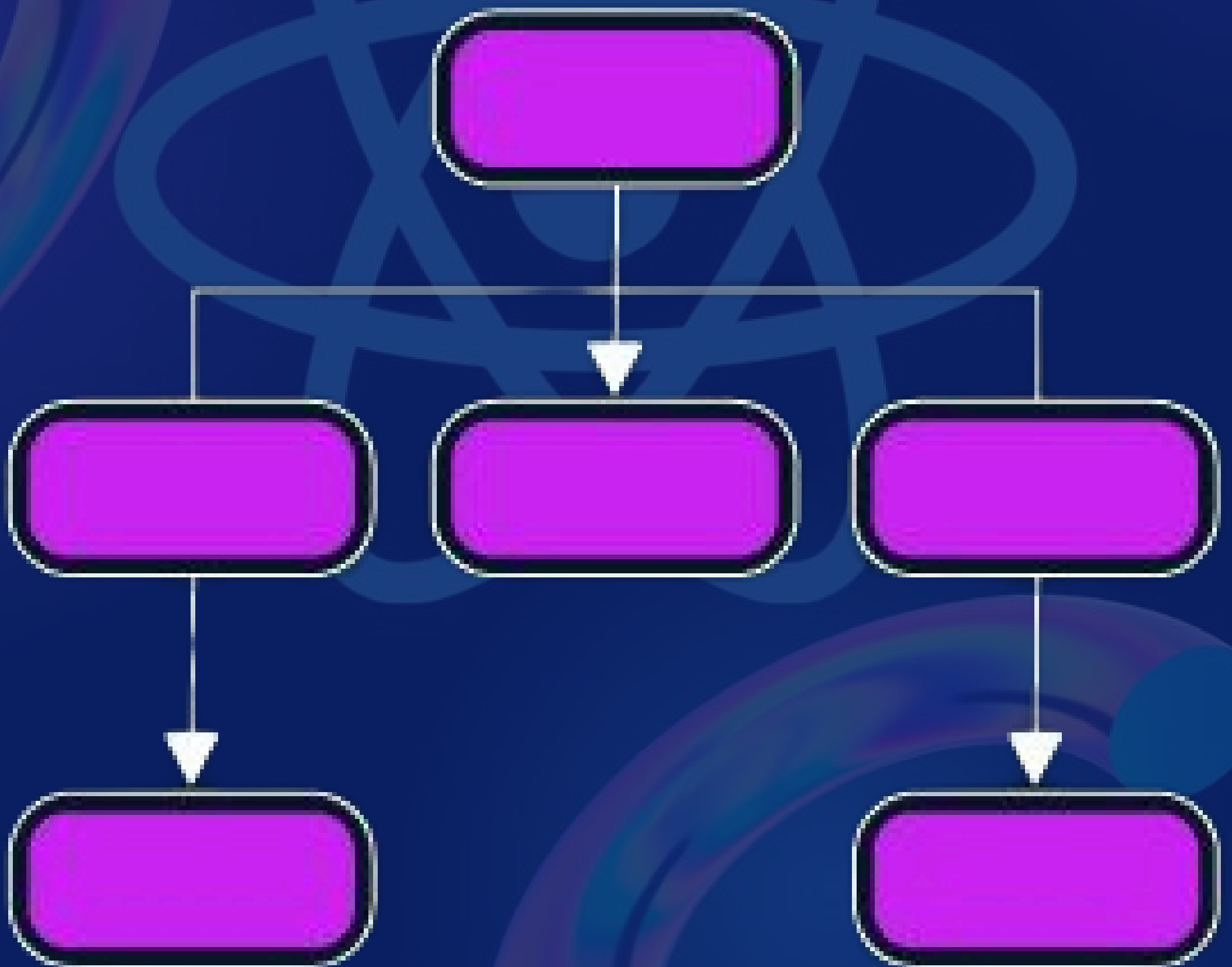


Day 21

Global State Management In React (Part 3)



The “All-mighty” Redux

Redux is a **predictable state container** for JavaScript apps. In React applications, it is commonly used to manage global state, which means **keeping state outside** of individual components and sharing it across the app.

While using **Context API + Reducers** for global state management, it is relatively basic and works best for **smaller applications** where global state management is not too complex, **Redux** is more **powerful** and **structured**, designed to handle **complex applications** with large state needs.

It is built around the concept of a **single source of truth** (a centralized store) and enforces a strict pattern of how state changes (through actions and reducers), which makes the data flow more predictable and maintainable in larger apps.

Redux enables centralized state management, where all state is managed in one place (the store), and changes to state follow a unidirectional data flow pattern, making your application more predictable and easier to debug.

Key Concepts of Redux

- **Store:** The central place where the state of your entire application lives.
- **Actions:** Descriptions of what should be done (e.g., fetching data, adding an item). Actions are plain JavaScript objects with a type field.
- **Reducers:** Pure functions that take the current state and an action as arguments and return a new state.
- **Dispatch:** A function used to send actions to the Redux store.
- **Selectors:** Functions used to extract data from the Redux store.

How Redux Works:

Redux operates based on a few principles:

- Single source of truth: The state of your whole application is stored in an object tree within a single store.
- State is read-only: The only way to change the state is to dispatch an action.
- Changes are made with pure functions: To specify how the state tree is transformed by actions, you write pure reducers.

Setting Up Redux In React

To show the steps involved in managing global state with Redux in a React project, let's walk through setting up Redux and using it to fetch data from an API.

There are 2 patterns of working with Redux in a React project, either by the **basic method** (separating the actions, states, reducers, and thunk in different files), or by directly installing the Redux full kit known as the Redux Toolkit, which has everything needed for your setup and you can declare all states, reducers, etc. in a single file.

Although, the first method is deprecated but still works, this is for the purpose of explanation only, **it is recommended that you use the Redux Toolkit method.**

The Basic Method

Step 1: Setting Up Redux with React

To begin, let's install the required packages for Redux and React integration.

In your terminal, navigate to your project's root folder (You must have created a React project already), and type in:

“**npm install redux react-redux**” like this:

```
kemil@DESKTOP-QKVDT62 MINGW64 /c/Desktop/My-Projects/my-vite-app  
● $ npm install redux react-redux
```

- **redux:** The core Redux library.
- **react-redux:** Bindings that allow React components to interact with Redux.

Step 2: Create a Redux Store

In Redux, the **store** holds the global state. We'll create a store using the **createStore** method from Redux.

For more organization, create a new folder in your **root folder**, named “**utils**”, and create a file in the utils folder named “**store.js**”

```
1 // /utils/store.js
2 import { createStore } from 'redux';
3 import rootReducer from './reducers';
4
5 const store = createStore(rootReducer);
6
7 export default store;
```

Here, we use **rootReducer**, which combines all the reducers in the application (more on reducers in a moment).

You will get a linting warning that the **createStore** method is deprecated, but don't worry about it, it will still work.

Step 3: Defining Actions

Actions in Redux are plain JavaScript objects that describe an event. You can think of them as “messages” that tell Redux what you want to do.

In your “**utils**” folder, create another folder named “**actions**”, in this folder, create a file named “**dataActions.js**”

```
1 // /utils/actions/dataActions.js
2 export const fetchDataRequest = () => ({
3   type: 'FETCH_DATA_REQUEST',
4 });
5
6 export const fetchDataSuccess = (data) => ({
7   type: 'FETCH_DATA_SUCCESS',
8   payload: data,
9 });
10
11 export const fetchDataFailure = (error) => ({
12   type: 'FETCH_DATA_FAILURE',
13   payload: error,
14 });
```

- **fetchDataRequest:** Indicates the start of the data fetching process.
- **fetchDataSuccess:** Fired when the data is successfully fetched.
- **fetchDataFailure:** Fired if there's an error fetching data.

Step 4: Creating a Reducer

Reducers specify **how the application's state changes in response to actions**. It takes the **current state** and **an action** and returns a **new state**, just like we learned yesterday.

In your “utils” folder, create another folder named “reducers”, in this folder, create a file named “dataReducers.js”

In your dataReducers.js file:



```
1 // /utils/reducers/dataReducer.js
2 const initialState = {
3   loading: false,
4   data: [],
5   error: '',
6 };
7
8 const dataReducer = (state = initialState, action) => {
9   switch (action.type) {
10     case 'FETCH_DATA_REQUEST':
11       return {
12         ...state,
13         loading: true,
14       };
15     case 'FETCH_DATA_SUCCESS':
16       return {
17         ...state,
18         loading: false,
19         data: action.payload,
20         error: '',
21       };
22     case 'FETCH_DATA_FAILURE':
23       return {
24         ...state,
25         loading: false,
26         data: [],
27         error: action.payload,
28       };
29     default:
30       return state;
31   }
32 };
33
34 export default dataReducer;
```

initial state of the reducers

each state is updated

In the file above:

- **Initial State:** loading, data, and error represent the states during the API call.
- **FETCH_DATA_REQUEST:** Sets loading to true while data is being fetched.
- **FETCH_DATA_SUCCESS:** Sets data when the API call is successful.
- **FETCH_DATA_FAILURE:** Updates the error if the API call fails.

Note that all “**case**” values are written in UPPER CASE letters

Step 5: Combine Reducers (if multiple reducers)

We often have **multiple reducers** in a large app. You can combine them using Redux's **combineReducers** method.



```
1 // /utils/reducers/index.js
2 import { combineReducers } from 'redux';
3 import dataReducer from './dataReducer';
4
5 const rootReducer = combineReducers({
6   data: dataReducer,
7 });
8
9 export default rootReducer;
10
```

You can import as many reducers as you have in your project, but for this project we only have dataReducers.

Step 6: Connecting Redux to React

To make the **Redux store** accessible to React components, we use the **Provider** component from react-redux.

You will have to wrap your entire application with the Provider component either in your “**index.js**” file (if using CRA method) or “**main.jsx**” file (if using Vite method) like this:



```
1 // src/index.js
2 import React from 'react';
3 import ReactDOM from 'react-dom';
4 import { Provider } from 'react-redux';
5 import App from './App';
6 import store from './store';
7
8 ReactDOM.render(
9   <Provider store={store}>
10     <App />
11   </Provider>,
12   document.getElementById('root')
13 );
```

Provider wrapping around the entire app

Provider: This makes the Redux store available to all components in your app.

This is a CRA method, so it's in index.js

@oluwakemi Oluwadahunsi

Step 7: Fetching Data from API using Redux and React

For this example, we will fetch a list of users from the JSONPlaceholder API.

Add this in your dataActions file:

```
1 // utils/actions/dataActions.js
2 export const fetchData = () => {
3   return async (dispatch) => {
4     dispatch(fetchDataRequest());
5     try {
6       const response = await fetch('https://jsonplaceholder.typicode.com/users');
7       const data = await response.json();
8       dispatch(fetchDataSuccess(data));
9     } catch (error) {
10      dispatch(fetchDataFailure(error.message));
11    }
12  };
13 };
```

Remember the actions functions we declared previously, the response from the endpoint, if successful is dispatched using the **fetchDataSuccess** functions and if it fails, the error is dispatched using the **fetchDataFailure** function.

This **fetchData** function uses **thunk middleware** to handle asynchronous logic. To use this, install **redux-thunk**

In your terminal, type: “**npm install redux-thunk**”

Thunk in Redux is a **middleware** that allows you to write action creators that return a function instead of an action object. This function can perform asynchronous operations (like API calls) and dispatch actions based on the result, enabling **side effects in Redux**.

Now, update the **store** to use **thunk**:



```
1 // /utils/store.js
2 import { createStore, applyMiddleware } from 'redux';
3 import thunk from 'redux-thunk';
4 import rootReducer from './reducers';
5
6 const store = createStore(rootReducer, applyMiddleware(thunk));
7
8 export default store;
```


Step 8: Create React Component to Display Fetched Data

Create a component named “UserList.jsx/jsx” in your “components” folder”

```
1
2 import { useEffect } from "react";
3 import { useDispatch, useSelector } from "react-redux";
4 import { fetchData } from "../actions/dataActions";
5
6 const UserList = () => {
7   const dispatch = useDispatch();
8   const { loading, data, error } = useSelector((state) => state.data);
9
10  useEffect(() => {
11    dispatch(fetchData());
12  }, [dispatch]);
13
14  return (
15    <div>
16      {loading && <p>Loading ... </p>}
17      {error && <p>Error: {error}</p>}
18      <div className="container">
19        {data.map((user) => (
20          <div key={user.id} className="wrapper">
21            <img
22              src={user.img}
23              alt="user-image"
24              title="user"
25              className="avatar"
26            />
27            <h2>{user.name}</h2>
28            <p>{user.role}</p>
29            <p>Phone: {user.phone}</p>
30            <address>
31              Address: {user.address.suite}, {user.address.street}, {user.address.city}.
32            </address>
33            <a href={`mailto:${user.email}`}>{user.email} </a>
34            <p>Website: {user.website}</p>
35          </div>
36        ))}
37      </div>
38    </div>
39  );
40 };
41
42 export default UserList;
```

- **useDispatch**: Dispatches the fetchData action when the component mounts.
- **useSelector**: Selects the loading, data, and error state from the Redux store.
- The **useEffect hook** is used to trigger the API call when the component renders.

Step 9: Putting it All Together

In your “App.js/jsx” file, import the UserList component:

```
1 // src/App.js
2 import React from 'react';
3 import UserList from './components/UserList';
4
5 function App() {
6   return (
7     <div>
8       <h1>Redux API Fetch Example</h1>
9       <UserList />
10    </div>
11  );
12 }
13
14 export default App;
```

Setup Using Redux Toolkit

Redux Toolkit (RTK) is the official, recommended way to write Redux logic. One of its key features is **slices**, which help to streamline the process of writing reducers, actions, and handling side effects.

In the example we used for the basic Redux to fetch data from an API, **slices** can simplify and organize our code by bundling action creators and reducers together.

In Redux Toolkit, a **slice** represents a "**slice of state**" and includes all the logic related to that state (reducers, actions, and initial state) in a **single file**.

Let's refactor our previous API fetching example using Redux Toolkit.

Step 1: Setting Up Redux Toolkit

First, install Redux Toolkit by typing this in your terminal (create a fresh project please):

“npm install @reduxjs/toolkit”

We no longer need to install **redux-thunk** separately, as **Redux Toolkit** includes **thunk** by default.

Step 2: Creating a Slice

We will create a slice to manage the global state for fetching users. This slice will automatically generate **action creators** and a **reducer**.

Create a **utils** folder as we did in the first example, and create another folder in it named **“slices”**. In the “slices” folder, create a file named **“userSlice.js”** like this:


```
1 // utils/slices/userSlice.js
2 import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
3
4 // Initial state for the slice
5 const initialState = {
6   loading: false,
7   data: [],
8   error: '',
9 };
10
11 // Async thunk to fetch users
12 export const fetchUsers = createAsyncThunk('users/fetchUsers', async () => {
13   const response = await fetch('https://jsonplaceholder.typicode.com/users');
14   const data = await response.json();
15   return data;
16 });
17
18 // Create the slice
19 const userSlice = createSlice({
20   name: 'users',
21   initialState,
22   reducers: {}, // You can add more reducers here if needed
23   extraReducers: (builder) => {
24     builder
25       .addCase(fetchUsers.pending, (state) => {
26         state.loading = true;
27       })
28       .addCase(fetchUsers.fulfilled, (state, action) => {
29         state.loading = false;
30         state.data = action.payload;
31         state.error = '';
32       })
33       .addCase(fetchUsers.rejected, (state, action) => {
34         state.loading = false;
35         state.data = [];
36         state.error = action.error.message;
37       });
38   },
39 });
40
41 export default userSlice.reducer;
```

initial state defined

async function to fetch users

reducers used
for updating
the states

In the `userSlice.js` file:

- **`createSlice`**: This function helps you write Redux logic by automatically generating **action creators** and a **reducer**.
- **`createAsyncThunk`**: Handles asynchronous actions (like API calls). It automatically dispatches the **pending**, **fulfilled**, and **rejected** actions based on the promise lifecycle.
- **`extraReducers`**: Handles actions generated by **`createAsyncThunk`**. When the API call is pending, it sets loading to true. If the call is successful, it sets the data with the fetched results. If it fails, it sets an error message.

Step 3: Setting Up the Redux Store

Next, let's configure the Redux store with the `userSlice` reducer.

Create a “store” file in your `utils` folder:

```
1 // src/store.js
2 import { configureStore } from '@reduxjs/toolkit';
3 import userReducer from '../features/userSlice';
4
5 const store = configureStore({
6   reducer: {
7     users: userReducer, // You can add other reducers here if needed
8   },
9 });
10
11 export default store;
```

Instead of using the deprecated **createStore** method in the old setup, we use Redux Toolkit's **configureStore**, and you don't need to worry about adding middleware like **thunk**, it's included by default.

Step 4: Connecting Redux Store to React

As before, we need to wrap the app with the **Provider** to give the Redux store access to the components (refer to step 6 in page 13) .

Step 5: Fetching Data and Using the Slice in a React Component

Now, let's update the React component (UserList.js/jsx) to fetch users using the **fetchUsers** **thunk** and display the data.

@oluwakemi Oluwadahunsi



```
1
2 import { useEffect } from "react";
3 import { useDispatch, useSelector } from "react-redux";
4 import { fetchUsers } from "../features/userSlice";
5
6 const UserList = () => {
7   const dispatch = useDispatch();
8   const { loading, data, error } = useSelector((state) => state.users);
9
10  useEffect(() => {
11    dispatch(fetchUsers());
12  }, [dispatch]);
13
14  return (
15    <div>
16      <h1>Users List</h1>
17      {loading && <p>Loading... </p>}
18      {error && <p>Error: {error}</p>}
19      <div className="container">
20        {data.map((user) => (
21          <div key={user.id} className="wrapper">
22            <img
23              src={user.img}
24              alt="user-image"
25              title="user"
26              className="avatar"
27            />
28            <h2>{user.name}</h2>
29            <p>{user.role}</p>
30            <p>Phone: {user.phone}</p>
31            <address>
32              Address: {user.address.suite}, {user.address.street}, {" "}
33              {user.address.city}. {" "}
34            </address>
35            <a href={`mailto:${user.email}`}>{user.email}</a>
36            <p>Website: {user.website}</p>
37          </div>
38        ))}
39      </div>
40    </div>
41  );
42 };
43
44 export default UserList;
```

In this userList.js file:

- **useDispatch():** Used to dispatch the **fetchUsers** action when the component mounts.
- **useSelector():** Selects the **loading**, **data**, and **error** from the Redux store's state managed by **userSlice**.
- **useEffect():** Calls the **fetchUsers** action as soon as the component is rendered.

Step 6: Putting Everything Together

Now, import the UserList component in your “App.js/jsx” file for output (refer to step 9 in page 17 above).

Do note that the component extension is **.js** for CRA method and **.jsx** for Vite method.

Now run your application to view the fetched data in your browser. You should see a list of user's data displayed.

I added some stylings to mine using Tailwind CSS. You can add yours too. Here is mine:

Users List



Leanne Graham

Phone: 1-770-736-8031 x56442

Address:

Apt. 556, Kulas Light, Gwenborough.

Sincere@april.biz

Website: hildegard.org



Ervin Howell

Phone: 010-692-6593 x09125

Address:

Suite 879, Victor Plains, Wisokyburgh.

Shanna@melissa.tv

Website: anastasia.net



Clementine Bauch

Phone: 1-463-123-4447

Address:

Suite 847, Douglas Extension, McKenziehaven.

Nathan@yesenia.net

Website: ramiro.info



Patricia Lebsack

Phone: 493-170-9623 x156

Address:

Apt. 692, Hoeger Mall, South Elvis.

Julianne.OConner@kory.org

Website: kale.biz



Chelsey Dietrich

Phone: (254)954-1289

Address:

Suite 351, Skiles Walks, Roscoeview.

Lucio_Hettinger@annie.ca

Website: demarco.info



Mrs. Dennis Schulist

Phone: 1-477-935-8478 x6430

Address:

Apt. 950, Norberto Crossing, South Christy.

Karley_Dach@jasper.info

Website: ola.org

Remember to add an image in your folder as there are no images from the endpoint.

Best Practices for Using Redux

1. Use Redux only for Global State:

If a piece of state is only used by one component or its immediate children, local state (using `useState`) is better.

2. Keep Reducers Pure:

Reducers should always be pure functions, returning new state without side effects.

3. Use Redux Toolkit:

It simplifies the syntax of Redux and reduces boilerplate.

4. Combine Thunks or Sagas:

For complex asynchronous logic, you can combine Redux with **redux-thunk** (for simple async logic) or **redux-saga** (for complex workflows).



I hope you found this material
useful and helpful.

Remember to:

Like

Save for future reference

&

Share with your network, be
helpful to someone 🙌

Hi There!

Thank you for reading through

Did you enjoy this knowledge?

 Follow my LinkedIn page for more work-life balancing and Coding tips.

 LinkedIn: Oluwakemi Oluwadahunsi

kodemaven-portfolio.vercel.app